

Some notes on calculations with mire based keratometric data

Purpose

The subject of the best curve or surface to represent data is extensive, but we'll separate it into **interpolation**, which assumes each data point is accurate, and seeks to find a smooth curve that passes through each point but also allows a good estimate of values at in-between points, and **least square fitting**, which allows for errors in the data points, and seeks a smooth curve that fits the data as well as possible without necessarily passing exactly through each point.

If some of the data points indicate a flat surface, for example, forcing a smooth curve that fits all the points may make the interpolation in-between oscillate, making the interpolation unreliable. Linear interpolation does not have this weakness, but does not provide smoothness. As has been pointed out greater smoothness does not mean greater accuracy with interpolation. Least square fitting sacrifices local exactness by prioritizing smoothness with accuracy globally.

The object of splining the Placido disk based data is to find a height function $z=z(x,y)=z(r,\theta)$ which smoothly passes through all of the data points. By smoothly, it is meant that the second derivatives of the function are continuous throughout. The height data may then be used to calculate all of the derivatives needed for intrinsic shape property calculations, lines of curvature, etc.. It should be recognized that there are inherent limitations to splining a surface with two independent one-dimensional splines which are beyond the scope of this discussion.

EyeSys data is given as two files of r and R vs θ where R is the sagittal curvature given in millimeters. The radius R is related to the curvature D in diopters by

$$R=(n-1)/D. \quad \text{Given} \quad z_r = \frac{\partial z}{\partial r} \quad \text{and} \quad z_r = \frac{-r}{\sqrt{R^2 - r^2}},$$

We are indirectly given the value of $z_r(r,\theta)$ at 16 points along each of 360 meridian directions (given every degree). Taking the values from the two files RA... and XX..., it is assumed that the values are given by $R=XX$ values $r=RA$ values at each degree, resulting in slopes using the above formula.

Atlas data is given in terms of corneal powers, by default assumed to be "axial" or "sagittal" power, defined by the above keratometric formula with $n=338.5$ which allows the "power" to be converted to a "curvature" which is actually a measure of the slope, not the curvature. In the Atlas data files this assumption leads to using $r=\text{DISTANCE}$ and $R=(n-1)/\text{POWER}$ at 22 or 25 points along each of 180 meridian directions (given every two degrees).

Assumptions and limitations:

Note that Placido disk, mire-based data assumes implicitly that rays stay in the meridian plane and that there is no "skew deviation". Although it is not commonly realized, the assumption of rays staying in the meridional plane with the surface normal in this same plane leads to the result that the surface studied must be axisymmetric. To quote SE Wilson and SD Klyce, who were probably the most cited in this field in the 1980s and 1990's "The magnitude of this error is as yet undetermined, although it is thought to be small."

A quick proof follows:

For the rays to stay in the plane, there are three vectors which must be coplanar: the VK axis along which the image is obtained, the surface normal at the reflection point and the meridian ray from the VK axis to the surface normal. In Cartesian coordinates these are as follows: $\{0,0,cz\}$, $\{-\text{grad } z,1\}$, $\{x,ax,bz\}$ where

a,b,c are constants. If these are coplanar then their triple scalar product must be zero:

$$\begin{vmatrix} 0 & 0 & cz \\ x & ax & bz \\ -\frac{\partial z}{\partial x} & -\frac{\partial z}{\partial y} & 1 \end{vmatrix} = 0 = -\frac{\partial z}{\partial y} + a \frac{\partial z}{\partial x} \quad . \quad \text{Note that the equation of the meridian}$$

(the middle vector) has the equation $y=ax$, which allows us to identify $a=\tan(\theta)$ the slope. Changing the Cartesian derivatives for their cylindrical equivalents with:

$$z_x = z_r \cos t - \frac{z_t}{r} \sin t$$

and substituting $a=\tan(t)$ gives the result that $z_t/r = 0$, which

$$z_y = z_r \sin t + \frac{z_t}{r} \cos t$$

is the condition of axisymmetry for $r>0$. QED

However, the above proof does not prevent us from assuming that the z_r above are close to the true radial derivatives and **possibly** calculating height data **fairly accurately** from them, according to the previously quoted authors.

Splining Methods

Boundary conditions must be chosen to uniquely define the cubic spline which passes through the points. We can either specify the derivative of the function to be splined along its boundaries or specify that the second derivative at the boundary is zero - the "natural" spline condition. For our splining of the cornea we chose to have natural spline conditions at the outer most mire ring, assuming that the corneal tangential curvature near the outermost mire is flat... No assumption need be made at the innermost mire since we pair each radial meridian with the mire points 180 degrees apart to form a diagonal mire across the center of the keratometric data where there is no data. Thus the EyeSys data for angle degree 1 is paired with the angle information at degree =181 to form a diagonally oriented spline with 32 data points. There are 180 such natural cubic splines formed through each set of 32 points. Atlas data is provided every two degrees with 22 or 25 rings, resulting in 90 splines with up to 50 points each.

Forming the meridian splines

We follow the method and notation of Press et al.: Linear interpolation of $z_i=z(r_i)$ is on an interval between r_j and r_{j+1} is written $z=Az_j+Bz_{j+1}$ with

$$B = \frac{r - r_j}{r_{j+1} - r_j}, A = 1 - B. \quad \text{Cubic spline interpolation is written with the cubic spline}$$

$z = Az_j + Bz_{j+1} + Cz_j'' + Dz_{j+1}''$ and C and D specified such that $z'' = Az_j'' + Bz_{j+1}''$. Clearly z is then a smooth function since the second derivative is continuous. The equation of continuity for the first derivative of $z(r)$ works out to the equation

$$\frac{r_j - r_{j-1}}{6} z_{j-1}'' + \frac{r_{j+1} - r_{j-1}}{3} z_j'' + \frac{r_{j+1} - r_j}{6} z_{j+1}'' = \frac{z_{j+1} - z_j}{r_{j+1} - r_j} - \frac{z_j - z_{j-1}}{r_j - r_{j-1}}, \quad \text{which form a tridiagonal system}$$

in z_j'' (see the cyclic system below with $a_1=0$ and $c_n=0$) which can be solved in $O[n]$ storage space and computation time. Boundary conditions are given by either setting the value of the first derivative of z at the endpoints or specifying that the second derivative at the endpoint is zero (the "natural spline").

Integrating the meridian splines

We then integrate the spline equation; it is a simple cubic polynomial after all, to obtain $z(r, \theta)$ at all points r along each of the 180 meridians. The

original spline is $z_r = \sum_1^4 c_i t^{i-1}$ where $t = \frac{r - r_j}{r_{j+1} - r_j}$ and the c_i are constants found for

each interval j to $j+1$. Then the heights are $z = c_0 + (r_{j+1} - r_j) \sum_1^4 c_i \frac{t^i}{i}$. The constants c_0 for each interval are found by applying continuity at r_j with

$c_0(k+1) = c_0(k) + (r_{k+1} - r_k) \sum_{i=1}^4 c_i(k) / i$. The initial $c_0(1)$ is chosen so that z at $r=0$ is the same for all the meridians. Note that after this integration we have a cubic spline in the derivatives in the meridian plane but a quartic equation in the heights.

Forming the periodic spline

Finally we form a periodic cubic spline to smoothly fit the meridians to one another. The periodic spline has neither derivatives at the boundaries specified nor natural spline conditions, but rather the condition set on the derivatives such that the function being splined is periodic. The spline equations for θ are identical to those for r given above, except that the boundary conditions are given by the periodic conditions shown below, with period 2π

$$\frac{\theta_n - \theta_{n-1}}{6} z_{n-1}'' + \frac{\theta_1 - \theta_{n-1} + 2\pi}{3} z_n'' + \frac{\theta_1 - \theta_n + 2\pi}{6} z_1'' = \frac{z_1 - z_n}{\theta_1 - \theta_n + 2\pi} - \frac{z_n - z_{n-1}}{\theta_n - \theta_{n-1}} \text{ and}$$

$$\frac{\theta_1 - \theta_n + 2\pi}{6} z_n'' + \frac{\theta_2 - \theta_n + 2\pi}{3} z_1'' + \frac{\theta_2 - \theta_1}{6} z_2'' = \frac{z_2 - z_1}{\theta_2 - \theta_1} - \frac{z_1 - z_n}{\theta_1 - \theta_n + 2\pi}.$$

Together with the spline equations these do not form a tridiagonal system since the periodic terms "wrap" the matrix coefficients. The non-zero elements of the matrix system to be solved for the second derivatives at each point are then:

$$\begin{bmatrix} b_1 & c_1 & . & .. & . & 0 & a_1 \\ a_2 & b_2 & c_2 & .. & . & . & 0 \\ . & . & . & .. & a_{n-1} & b_{n-1} & c_{n-1} \\ c_n & 0 & . & .. & 0 & a_n & b_n \end{bmatrix} \begin{bmatrix} z_1'' \\ . \\ . \\ . \\ z_n'' \end{bmatrix} = \begin{bmatrix} d_1 \\ . \\ . \\ . \\ d_n \end{bmatrix} \text{ i.e. tridiagonal except for two off-}$$

diagonal terms, where the coefficients vectors a, b, c, d are inferred from the spline equations and boundary conditions given above. Solving this system without taking advantage of its sparse nature would consume $O[n^2]$ in storage space and computation (e.g. with elimination methods). Press et al. gives an algorithm using the Sherman-Morrison formula with the usual tridiagonal algorithm also solving the "cyclic tridiagonal system" in $O[n]$ time and storage space.

Alternatively, the system can also be solved in $O[n]$ with the following algorithm. Starting with the 1st and nth equations and working inwards to the 2nd and n-1st and so on, for the j th pair of equations we can write:

$$\begin{bmatrix} b_j & 0 \\ 0 & b_{n-j+1} \end{bmatrix} \begin{bmatrix} z_j'' \\ z_{n-j+1}'' \end{bmatrix} + \begin{bmatrix} a_j & 0 \\ 0 & c_{n-j+1} \end{bmatrix} \left\{ \tilde{A}_{j-1} \begin{bmatrix} z_j'' \\ z_{n-j+1}'' \end{bmatrix} + \vec{v}_{j-1} \right\} = \begin{bmatrix} d_j \\ d_{n-j+1} \end{bmatrix} - \begin{bmatrix} c_j & 0 \\ 0 & a_{n-j+1} \end{bmatrix} \begin{bmatrix} z_{j+1}'' \\ z_{n-j}'' \end{bmatrix}$$

$$\text{with } \begin{bmatrix} z_{j-1}'' \\ z_{n-j+2}'' \end{bmatrix} = \left\{ \tilde{A}_{j-1} \begin{bmatrix} z_j'' \\ z_{n-j+1}'' \end{bmatrix} + \vec{v}_{j-1} \right\} \text{ for } j > 1 \text{ and } \tilde{A}_0 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \vec{v}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \text{ for } j=1. \text{ Each pair}$$

is solved for in terms of the inner next pair of values. Using the first equation with $j=1$ yields the result for the second equation for $j=2$ by solving a 2×2 system of equations. Using the matrix A_1 and vector v_1 , the process is iterated

repeatedly almost $n/2$ times. At the center of the matrix we have to solve either a 2×2 system of equations if n is even or a 3×3 system if n is odd. If n is even the final equation becomes, with $j=n/2$:

$$\begin{bmatrix} b_{n/2} & 0 \\ 0 & b_{n/2+1} \end{bmatrix} \begin{bmatrix} z''_{n/2} \\ z''_{n/2+1} \end{bmatrix} + \begin{bmatrix} a_{n/2} & 0 \\ 0 & c_{n/2+1} \end{bmatrix} \left\{ \tilde{A}_{n/2-1} \begin{bmatrix} z''_{n/2} \\ z''_{n/2+1} \end{bmatrix} + \vec{v}_{n/2-1} \right\} = \begin{bmatrix} d_{n/2} \\ d_{n/2+1} \end{bmatrix} - \begin{bmatrix} c_{n/2} & 0 \\ 0 & a_{n/2+1} \end{bmatrix} \begin{bmatrix} z''_{n/2+1} \\ z''_{n/2} \end{bmatrix} \text{ which}$$

is a 2×2 system solvable for the middle two second derivatives. If n is odd:

$$\begin{bmatrix} b_j & c_j & 0 \\ a_{j+1} & b_{j+1} & c_{j+1} \\ 0 & a_{j+2} & b_{j+2} \end{bmatrix} \begin{bmatrix} z''_j \\ z''_{j+1} \\ z''_{j+2} \end{bmatrix} = \begin{bmatrix} d_j \\ d_{j+1} \\ d_{j+2} \end{bmatrix} + \begin{bmatrix} a_j & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & c_{j+2} \end{bmatrix} \begin{bmatrix} z''_{j-1} \\ 0 \\ z''_{j+3} \end{bmatrix} \quad \square \square$$

$$\begin{bmatrix} z''_{j-1} \\ z''_{j+3} \end{bmatrix} = \tilde{A}_{j-1} \begin{bmatrix} z''_j \\ z''_{j+2} \end{bmatrix} + \vec{v}_{j-1}, j = \frac{n-1}{2}$$

Once the center values are known the stored matrices A_j and vectors v_j are used for backsubstitution. We therefore solve the n -order system by storing the $4n$ coefficients and left hand side, solving $n/2-1$ 2×2 systems and one additional 2×2 or 3×3 system. We also need to store $n/2$ 2×2 matrices and length 2 vectors, overall a $O[n]$ storage and computation. Unlike the Sherman-Morrison algorithm, this one can be extended to greater than tridiagonal cyclic banded matrices.
<https://github.com/mostlyharmlessone/cyclic-banded-matrix>

Numerical derivatives may be taken from the above spline.

Least Square Fitting Methods

The subject of least square fitting is extensive, but we'll try to be brief. Here we will make a distinction between least square fitting to a single function for the entire data set, or piecewise continuous least square fitting with a set of basis functions.

As an example, if we had a periodic function, we could choose a truncated Fourier series with N parameters (the Fourier coefficients) and perform a least square fit. Depending on the function and number of terms the fit could theoretically be quite good, with the (possible) benefit of providing the Fourier coefficients, to which some meaning could be attached. A non-periodic function would end up with wild oscillations near the endpoints, the Gibbs phenomenon, and a truncated power series might be a better choice in this case.

A different approach is a piecewise continuous representation. Here, the data set is divided into intervals and each interval is fit separately, with the requirement that each interval's function's endpoints match the endpoints of the adjacent interval. In order to provide smoothness, ie with a continuous second derivative at each interval breakpoint, a basis function must be picked that is twice differentiable, for example with the choice of a cubic spline passing through the breakpoints (or "knots"). Note that if the knots are defined as the data points, we have the situation with cubic spline interpolation previously described. If the number of knots is less than the data points and at different locations, then we can find the cubic spline interpolation, passing through the knots, which is the best least square fit to the data set. If there no knots, we have the situation described above with a global (cubic polynomial) fit for all of the data points.

Different approach to piecewise C^2 continuous spline can be taken for convenience of computation: bspline, represented by four data points, or cubic spline representation with two data points and two second derivatives, or with two data points and two corresponding first derivatives.